The Second Brain

Philosophy, Technology, and the Making of an Externalized Mind

A deep explanation of why second brains exist, why they keep failing, and why — for the first time in the history of personal knowledge management — they finally work.

Date: April 2026 **Prepared for:** Phil Mora **Subject:** The philosophy and technology behind LLM-native personal knowledge management

---

Preface

You are building a second brain. That phrase means many things to many people — a note-taking app, a productivity system, a philosophy, a lifestyle, a religion. This document is not about any of those. It is about something more specific and more ambitious: **an externalized cognitive system that works with you, maintained by a large language model, structured as a human-readable markdown wiki, designed to compound in value over a lifetime.**

The pattern is due, in its modern form, to Andrej Karpathy — the former Director of AI at Tesla, co-founder of OpenAI, and one of the clearest thinkers about what large language models actually are. In late 2025 and early 2026, Karpathy stopped writing code with AI and started building knowledge with it. He documented the pattern in a GitHub gist and a series of tweets. What he described is old in spirit — it descends from Vannevar Bush's 1945 Memex, Doug Engelbart's augmenting-the-intellect research at SRI, Ted Nelson's hypertext dreams, Niklas Luhmann's physical Zettelkasten, and Andy Matuschak's evergreen notes — but it is fundamentally new in mechanism.

This document is a deep explanation of why that pattern works, why every previous attempt failed, and what the philosophical and technical implications are. It is structured as follows:

- **Part I** — The problem: why personal knowledge management has always failed
- **Part II** — The intellectual lineage: what we learned from Bush, Engelbart, Luhmann, Matuschak, Forte, and Appleton
- **Part III** — The Karpathy synthesis: what changed, and why it works now

- **Part IV** — The three-layer architecture in detail
- **Part V** — The four memory types and how they map to a wiki
- **Part VI** — The technology stack, layer by layer
- **Part VII** — The ingest-query-lint operational loop
- **Part VIII** — Why this works now and didn't before
- **Part IX** — Comparison to alternatives
- **Part X** — The compounding value thesis
- **Part XI** — Open questions, critiques, and frontiers

If you only read one part, read Part III. If you read two, add Part X.

Part I — The problem: why personal knowledge management has always failed

The paradox of personal knowledge

Every knowledge worker has the same experience: you read something genuinely valuable, you have a clear insight, and six months later you cannot find it, cannot reconstruct it, and often cannot remember whether it was real. The knowledge was encoded in your head briefly and then leaked out. Whatever you wrote down sits in a notebook, a Notion page, a Notes file, an email you sent yourself, or a bookmark you never reopened — all indexed by the wrong keys, all separated from the context that made them meaningful.

This is the knowledge paradox: **individual humans are capable of remarkable synthesis across material they have encountered, but the encounter is the bottleneck.** Human working memory is 4–7 items. Human long-term memory is gigantic but hard to retrieve from. Human retrieval is trigger-based — we need the right cue at the right moment. We cannot query our own knowledge.

The dream of personal knowledge management is to solve this. To create an external system that remembers what we cannot hold, retrieves what we cannot cue, and connects what we cannot reach from the inside.

Why traditional note-taking fails

Every traditional approach — paper notebooks, digital notes apps, bookmark managers, highlight tools, read-later services — fails at the same point: **maintenance cost grows faster than retrieval value.**

The first note is free. The second note is free. By the hundredth note, the act of writing the note is still free, but the act of making the note *findable* has begun to require work. You need to know where to put it. You need to tag it. You need to link it to other notes. You need to occasionally prune and reorganize.

By the thousandth note, maintenance requires a weekend a month. Most people never get there because they give up earlier, leaving behind a cemetery of half-populated Notion pages, Evernote tags, and “inbox” folders that never got processed.

The reason this happens is structural. Human brains are bad at bookkeeping. Good at ideas; bad at cross-references. Good at insights; bad at keeping the index current. The knowledge gets richer, but the infrastructure supporting that knowledge decays faster than the knowledge accumulates.

Why chat-with-your-docs products fail

The recent generation of AI products — "chat with your docs," "RAG-based knowledge search," "AI-powered second brains" — pitched themselves as the solution. The pitch: dump all your documents into our vector database, and the AI will answer any question.

This approach fails for a subtle but fatal reason: **retrieval returns chunks, not context.** When you ask a question, the system embeds your query, finds the top-k most similar chunks by cosine similarity, and dumps those chunks into the model's prompt. The model answers from the chunks.

The chunks are disconnected. Their relationships are invisible. The structural knowledge — which document they came from, what came before, what they reference — is stripped away by the chunking process. The model is given fragments and asked to synthesize; it often does, but the synthesis is shallow because the fragments are disconnected.

Worse: the knowledge base is opaque to you. You cannot read it. You cannot edit it. You cannot improve it. The embeddings are not human-readable. The system gets worse over time as stale content accumulates in the vector store, because you cannot tell what is stale without reading every document.

This is why no chat-with-your-docs product has ever produced a sustained second brain for anyone. They are useful for one-shot question answering against a fixed corpus. They are useless for a living, compounding knowledge system.

Why Notion/Roam/Logseq get partial credit

A second generation of tools — Notion, Roam Research, Logseq, Obsidian — improved on traditional note-taking in meaningful ways. Block-level references. Backlinks. Graph views. Databases. These are genuine advances.

But they still leave the maintenance problem unsolved. The user is still the maintainer. The user must still remember to add cross-references. The user must still write the one-line summary. The user must still notice that two notes are saying contradictory things. The user must still prune. In a complex knowledge base, the user will fail at some of these, and the failures compound.

Roam Research, at its peak, illustrated the problem. Users reported spending an hour a day "tending their graph" — the compounding, densely-linked knowledge network they had built. The tending was intellectually pleasant. It was also the reason most users quit after eighteen months. The graph had become an obligation.

The root problem

The root problem is that a living knowledge base requires two kinds of labor:

1. **Creative labor** — reading, thinking, synthesizing, writing new content
2. **Clerical labor** — indexing, cross-referencing, summarizing, pruning, maintaining consistency

Humans want to do (1). Humans do not want to do (2). Every personal knowledge management system in history has failed because it asks the human to do both, and the clerical labor grows super-linearly with knowledge base size.

The fundamental insight of the modern second brain is that **large language models are exceptionally good at clerical labor on text, and they do not get bored.** This changes the equation.

Part II — The intellectual lineage

The second brain did not begin in 2025. It has a century of intellectual history behind it. Understanding that lineage is essential — not as academic decoration, but because every current design decision has a reason rooted in a prior idea.

Vannevar Bush and the Memex (1945)

In July 1945, as the Second World War ended, Vannevar Bush — director of the U.S. Office of Scientific Research and Development — published an essay called *As We May Think* in *The Atlantic Monthly*. He proposed a device he called the **Memex**.

The Memex was a desk-sized microfilm reader augmented with the ability to create "associative trails" between documents. Bush's key insight was that **human thought proceeds by association, not by hierarchical classification,** and therefore a knowledge system should support associative trails rather than forcing everything into folders or indexes.

> *"The human mind operates by association. With one item in its grasp, it snaps instantly to the next that is suggested by the association of thoughts, in accordance with some intricate web of trails carried by the cells of the brain."*

Bush's Memex was never built. It was too ambitious for 1945's technology. But it established two enduring principles:

1. **Associative linking is primary.** Hierarchical organization is a special case, not the default.
2. **The trail matters as much as the document.** What you connect is as important as what you collect.

Every subsequent personal knowledge system — from hypertext to wikis to Roam — is arguably a realization of the Memex.

Doug Engelbart and augmenting the intellect (1962)

Doug Engelbart, at SRI, built what Bush imagined. His 1962 report, *Augmenting Human Intellect: A Conceptual Framework*, proposed that computers could amplify human cognition rather than merely automate computation. His 1968 "Mother of All Demos" introduced — in a single 90-minute demonstration — the mouse, hypertext, real-time collaboration, video conferencing, and the concept of a computer as a cognitive partner.

Engelbart's key contribution to the second brain lineage was the **co-evolution thesis**: the human and the augmentation system must evolve together. Neither is static. The system changes how the human thinks; the human changes how the system is used; both grow together.

This thesis is now central to the LLM-native second brain. You are not building a finished artifact. You are building a living system that will change how you think, and your thinking will change what the system becomes.

Ted Nelson and hypertext (1965)

Ted Nelson coined the word "hypertext" in 1965. His Project Xanadu was a never-quite-finished vision of a universal hypertext system with two-way links, transclusion (a document embedding part of another), and version control.

Nelson's contribution: **links are bidirectional, and embedded references should stay live.** The modern wiki's `[[backlinks]]` are a direct descendant of Nelson's two-way links. When you create a new `[[concept]]` reference, both directions of the connection should be traversable: from A to B, and from B to A.

Niklas Luhmann and the Zettelkasten (1952–1998)

Niklas Luhmann, a German sociologist, wrote 70 books and over 400 academic articles over his lifetime. He attributed his productivity to a physical filing cabinet system he called the **Zettelkasten** — the slip-box. By his death, the Zettelkasten contained roughly 90,000 index cards.

Luhmann's system was composed of three rules:

1. **Atomic notes.** One idea per card. No card contains two ideas. If two ideas belong together, they get two cards plus a reference.
2. **Unique identifiers.** Every card had an arbitrary but unique ID (not a topic label — just a sequence number). This meant the card could be referenced without changing its identity over time.
3. **Dense linking.** Every card referenced related cards. The value of the system was in the connections, not in the cards themselves.

Luhmann's fourth, unwritten, rule: **write in your own words.** He refused to quote others at length. Every idea he encountered was paraphrased in his own voice. The act of rephrasing was where understanding happened.

This is the single most important principle inherited by the modern second brain. It is the rule that separates a knowledge base from a bookmark folder. Paraphrasing is not a decoration — it is the mechanism of understanding.

Tiago Forte and Building a Second Brain (2022)

Tiago Forte's *Building a Second Brain* popularized personal knowledge management for a non-academic audience. Forte's key contributions were methodological, not architectural:

- **PARA** — organize by actionability. Projects (deadlines, outcomes), Areas (ongoing responsibilities), Resources (reference material), Archive (inactive). Hot content at the top; cold content at the bottom.
- **CODE workflow** — Capture, Organize, Distill, Express. The four-phase rhythm of processing new information.
- **Progressive summarization** — layered distillation. Save raw content; bold the important parts; highlight the most important parts of the bolded; write an executive summary; apply the insight.

Forte's PARA and progressive summarization are present, in adapted form, in every modern LLM-native second brain. PARA appears as the optional top-level organization (`projects/`, `areas/`, `resources/`, `archive/`). Progressive summarization appears as the ingest pipeline's multi-layer processing (raw file → paraphrased note → distilled concept).

Andy Matuschak and Evergreen Notes (2019–present)

Andy Matuschak, an interface researcher and former Apple engineer, articulated the "evergreen notes" concept — notes designed to improve over time. His five principles are the quality bar for modern second brains:

1. **Notes should be atomic.** One concept per note.
2. **Notes should be concept-oriented.** Named by the idea, not by the source. A note is `concepts/attention-mechanism.md`, not `notes-on-chapter-3.md`.
3. **Notes should be densely linked.** Every note has many cross-references; links are how meaning accumulates.
4. **Prefer associative ontologies to hierarchical.** Tags and links over folders.
5. **Write notes for yourself by default.** Clarity over formality.

Matuschak's notes system is publicly readable at notes.andymatuschak.org. Studying his actual notes is instructive: each is short, self-contained, extensively linked, and written in clear personal voice. This is what "evergreen" looks like in practice.

Maggie Appleton and Digital Gardens (2020–present)

Maggie Appleton, a designer and anthropologist, articulated the "digital garden" metaphor. The garden is a personal knowledge collection that is:

- **Messy in public.** Not everything is polished; notes are at different stages of maturity.
- **Non-linear.** No imposed narrative; readers enter where they want.
- **Always growing.** The garden is never finished. Maturity is a continuum: seedlings → buds → evergreens.

Appleton's contribution to the second brain is permission. A second brain does not have to be a finished book. It does not have to be organized before you start. It can be embarrassing at its edges. It can be published partially, or not at all. Growth and imperfection are features, not bugs.

Part III — The Karpathy synthesis: what changed

By 2025, the intellectual prerequisites for the modern second brain were all present. The pattern was waiting for a technological catalyst. That catalyst was the large language model with long context.

Three things Karpathy saw that others missed

Andrej Karpathy had been building and training LLMs for a decade when, in late 2025, he began publicly writing about how he was using them personally. His observations crystallized around three insights that cut through the hype.

Insight 1 — Context is working memory, not storage

Karpathy's LLM-as-OS framing, which he had been developing since late 2023:

- **LLM = CPU** — the processor that executes
- **Context window = RAM** — the fast, scarce working memory
- **Filesystem = disk** — the slow, abundant storage
- **Agents = long-running applications**

This framing sounds like a metaphor but is operationally precise. It means: **do not treat the context window like storage.** Do not pre-load it with everything you might need. Fill it with what the current operation actually requires, nothing more.

Every naive knowledge system violates this principle. They dump entire document corpora into the context window and hope for the best. The LLM, fighting against context rot (the next insight), performs worse than if it had been given only the relevant material.

Insight 2 — Context rot

Empirically, LLM performance degrades as the context window fills. The 10,000th token is less trustworthy than the 10th. Instruction-following weakens. Recall drops. Reasoning gets sloppier.

This has been measured. Anthropic, Chroma, and several other research groups have published benchmarks showing the degradation curve. It is not a catastrophic cliff — it is a gradual drift — but it is real and predictable.

The implication: **pre-loading "just in case" actively harms performance on the current task.** The context window is a scarce resource. Every token you spend on material the LLM doesn't need is a token stolen from the quality of the LLM's current work.

Insight 3 — Compile, don't re-derive

Every retrieval-augmented system treats each query in isolation. The query comes in; chunks are retrieved; the model answers; everything is discarded. Nothing persists. The effort spent on one query doesn't help the next.

Karpathy's counter-proposal: **compile the knowledge once.** When a new source arrives, the LLM reads it, extracts concepts, writes them into the wiki, cross-references them to existing material, updates the index, updates the log. The compilation work happens once; every future query benefits from it.

This is the difference between a query-time system (RAG) and an ingestion-time system (the LLM wiki). The LLM wiki pays its cost upfront and amortizes it across all future uses.

The radical move: no RAG

The most counterintuitive conclusion Karpathy drew from these three insights:

> ***"No RAG. The LLM reads its own index."***

This is radical because RAG has been the default approach to knowledge augmentation for LLMs for three years. Every enterprise AI product, every "chat with your docs" startup, every internal corporate wiki-search system has been built on RAG.

Karpathy's argument:

- Modern long-context models can handle 200,000 to 1,000,000 tokens
- A typical personal wiki is 100–500 atomic notes, each a few hundred words
- Total size: 50,000 to 400,000 words — comfortably within 1M context
- Therefore: load what's needed directly; skip the retrieval layer entirely

When a query comes in, the LLM reads the wiki's index file (one-line summaries of every page), decides which full pages to read based on the query, reads those full pages, and answers. Retrieval happens at the **file level**, not the chunk level. And the LLM itself performs the retrieval — no vector database, no similarity search, no top-k chunks.

This is philosophically important. It means **the wiki remains human-readable and human-editable.** Every page is a markdown file. You can open it in any text editor. You can edit it. You can version control it. You can grep it. The LLM does not hide behind an opaque embedding space. The knowledge is in plain text, maintained by the LLM, available to both of you.

The self-improving loop

The final move that makes the pattern work: the LLM is not just a reader of the wiki, it is a **maintainer**.

The loop:

1. You capture a source (paper, article, meeting notes) into `raw/`
2. The LLM reads the source, identifies concepts and entities, and writes or extends wiki pages in your voice
3. You review what the LLM wrote and approve, refine, or reject
4. Over time, the LLM notices patterns — gaps where a referenced concept has no dedicated page, contradictions between old and new notes, orphaned pages
5. The LLM proposes maintenance actions; you approve
6. The wiki becomes better structured as it grows, not staler

This is the **lint step** — periodic health checks performed by the LLM. Without linting, wikis rot. With LLM-driven linting, they self-improve.

The loop is human-in-the-loop by design. The LLM proposes; you approve. You retain epistemic authority. The LLM handles the clerical labor it is actually good at, and does not attempt the creative labor that is yours.

Why Karpathy and not someone else

It is reasonable to ask why this synthesis fell to Karpathy rather than to the hundred-odd PKM thinkers who had been working adjacent to these problems for years. Two reasons.

First, Karpathy had a rare combination: deep technical understanding of what LLMs are mechanically doing, sustained personal investment in PKM as a user (not just a theorist), and a clear writing style that made the synthesis legible.

Second — and more structurally — the pattern required long-context models. Those arrived in production form in late 2024 (Claude 3.5 Sonnet's 200K context) and became routine in 2025 (Claude Opus 4 and Gemini 2.5 with 1M context). The pattern Karpathy proposed was not possible in 2022. In 2025, it became obvious to anyone paying attention.

The lesson is not that Karpathy is singularly brilliant (he is, but that is not the point). The lesson is that the conditions for this pattern arrived only recently, and we are in the first year of its practical viability.

Part IV — The three-layer architecture

The Karpathy pattern decomposes the system into three distinct layers. Each layer has a different editor, a different lifecycle, and a different purpose. Confusing the layers is the most common implementation error.

Layer 1 — Raw Sources

What lives here: everything imported from outside the system. Research papers, articles, PDFs, meeting transcripts, clipped web pages, book excerpts, email threads, Slack exports, voice note transcriptions, conversation logs, anything you did not create yourself.

Who can write it: only you (via capture). Once a source is in `raw/`, it is immutable.

Who reads it: the LLM during ingestion. After processing, the raw source is archived — moved to `raw/processed/{date}/` — and is read only if you or the LLM specifically needs to re-check against the original.

Why it exists as a layer: two reasons. First, provenance — every claim in the wiki should trace back to a source, and the sources must be preserved. Second, future re-ingestion — as your wiki's structure evolves, you may want to re-process old sources under new conventions, and you cannot do that if you deleted them.

Layer 1 is the least glamorous layer. It is also the most tempting to skip. You should not skip it.

Layer 2 — The Wiki

What lives here: the curated, structured, LLM-maintained knowledge base. Markdown files organized by memory type. Atomic notes. Dense cross-references. Written in your voice (via LLM paraphrasing).

Who can write it: both you and the LLM. Neither owns it exclusively. The LLM writes during ingestion and during query-time promotion; you write when inspiration strikes or when you disagree with what the LLM produced.

Who reads it: the LLM on every query. You, whenever you are curious. Both of you treat it as the authoritative knowledge layer.

The structure:

```
wiki/
├── README.md            ← schema (Layer 3, lives here conventionally)
├── index.md             ← MANDATORY content catalog
├── log.md               ← MANDATORY operation log
├── concepts/            ← atomic notes on ideas (the largest bucket)
├── entities/            ← notes on people, teams, organizations, products
├── sources/             ← notes ABOUT specific papers, books, articles, talks
├── projects/            ← active projects
├── areas/               ← ongoing areas of responsibility (PARA-style)
├── MOCs/                ← Maps of Content (hub pages linking related notes)
├── sessions/            ← episodic log of what you did and learned
├── sandbox/             ← working memory, half-formed ideas
└── raw/                 ← Layer 1 staging (and processed/ archive)
```

Most of the wiki lives in `concepts/`, `entities/`, and `sources/`. These three are the semantic memory layer — the "what I know about" part. A healthy wiki has these three folders growing together: every source produces a source note, which contributes facts to concept and entity notes.

`projects/` and `areas/` hold project-level and responsibility-level content (PARA-hot and PARA-warm). `MOCs/` are curated hub pages that link related content — useful for topics that have grown past 10–15 atomic notes.

`sessions/` is the episodic memory — what you did this week, what you learned, what's on your mind. Usually append-only short-term (`current-week.md`), compressed weekly into `weekly/YYYY-W##.md`.

`sandbox/` is the working memory — a safe place for drafts that are not yet ready to be atomic notes. You can delete things from here freely.

Layer 3 — The Schema

What lives here: the rules. `wiki/README.md` at the root of the wiki is the constitution. It specifies the directory structure, the citation conventions, the folder purposes, the writing rules, the tasks that operate on the wiki, and the privacy rules.

Who can write it: you, primarily. The LLM may propose schema changes but does not execute them without approval.

Who reads it: the LLM on every operation. It is always-loaded in every session, because it is the operating manual for the wiki.

Why the schema exists: it is the wiki's immune system. Without a written schema, conventions drift. Two ingestions months apart produce different note structures. The citation style changes. The folder purposes blur. The schema is how you, through a present act of legislation, bind all future ingestions to consistent standards.

The schema also co-evolves. You update it when you discover a convention that works better than the one you had. The co-evolution is intentional: a wiki that is not evolving is not learning.

The two mandatory navigation files

Karpathy's gist calls out two files at the root of the wiki as **non-optional**. They are not in a subfolder. They are at the top level because they are always read.

`wiki/index.md` — the content catalog

An auto-maintained flat list of every page in the wiki, with one-line summaries. The LLM reads this file first when starting to answer a query; based on the one-line summaries, it decides which full files to read.

Example skeleton:

```
# Work Brain — Index

*Auto-maintained by lint task. Last updated: 2026-04-09*

## Concepts
- [[concepts/distributed-tracing]] — Observability pattern for microservices
- [[concepts/context-engineering]] — Discipline of filling the LLM context window
- [[concepts/progressive-summarization]] — Layered distillation technique
- ...

## Entities
- [[entities/team-platform]] — Internal platform team
- [[entities/vendor-datadog]] — APM vendor, contract expires Q1 2027
- ...

## Sources
- [[sources/effective-context-engineering-anthropic]] — Sept 2025 foundational guide
- [[sources/dapper-paper]] — Google's 2010 tracing paper
- ...

## Projects
- [[projects/q2-platform-migration]] — Active, target 2026-06-30
- ...
```

The index is how the wiki becomes navigable at scale. A wiki of 50 files does not need an index; the LLM can read every file. A wiki of 500 files absolutely requires one — reading every file wastes context; reading the index first lets the LLM be surgical.

The index is regenerated by the lint task. Humans rarely edit it by hand.

`wiki/log.md` — the append-only operation log

Every operation that modifies the wiki — ingest, query (if promoted), lint, manual edit — gets a timestamped entry appended to the top of `log.md`. Never edit past entries. Parseable prefixes let the LLM search the log by operation type.

Example:

```
# Work Brain — Operation Log

*Append-only. Newest entries at the top. Never edit past entries.*

---

## [2026-04-09 14:32] lint | Weekly maintenance
- Compressed 14 session entries into sessions/weekly/2026-W14.md
- Flagged 3 orphan files for review
- Regenerated index.md
- Found 2 broken [[wikilinks]] (listed for review)

## [2026-04-09 09:15] ingest | Paper: Dapper distributed tracing
- Read raw/dapper-paper.pdf
- Created concepts/distributed-tracing.md (new atomic note)
- Extended concepts/observability.md with tracing subsection
- Created entities/google-research.md
- Created sources/dapper-paper.md
- Moved raw file to raw/processed/2026-04-09/

## [2026-04-08 17:42] query | "How does progressive summarization work?"
- Read concepts/progressive-summarization.md
- Read sources/building-a-second-brain.md
- Answered in conversation
- Promoted synthesized answer to concepts/progressive-summarization.md "Practical
workflow" section

---
```

The log's format is not ornamental. The parseable prefix `## [YYYY-MM-DD HH:MM] operation | short-description` lets both humans and LLMs search the log programmatically. "When did I first encounter this concept?" becomes a `grep`. "What did the lint task do last Sunday?" is instantly retrievable.

The log is the wiki's audit trail. It is also — subtly — a compounding asset of its own. The pattern of which concepts get queried most often, which get extended repeatedly, which go untouched for months, emerges from the log. Over time, you can analyze your own knowledge work.

Part V — The four memory types

Every piece of information a wiki holds belongs to exactly one of four memory types. These types are inspired by cognitive science — procedural, semantic, episodic, working — but reinterpreted for LLM-native systems. Mixing them is the single biggest source of wiki rot.

Procedural memory — HOW

Contents: rules, protocols, workflows, hard constraints, how-to procedures. The executable part of knowledge: things that produce behavior.

Examples in a work wiki: - "My decision framework for accepting a new project" - "How I run a retrospective" - "The questions I ask when evaluating a vendor" - "My weekly review checklist"

Lifecycle: slow-changing. Updates when something forces revision (a tool changes, a lesson is learned, a policy shifts). Rarely deleted.

Where it lives: `concepts/workflows/` or inline in `concepts/`, depending on whether you want to distinguish workflows from general concepts.

Distinguishing property: procedural memory is about action. If you extracted just the procedural files from your wiki, you would have an operations manual for yourself.

Semantic memory — WHAT

Contents: atomic facts about concepts, entities, and sources. The largest memory type; ~80% of a mature wiki lives here.

Three sub-categories:

1. **Concepts** (`concepts/`) — ideas. `distributed-tracing`, `context-engineering`, `progressive-summarization`. Named by the idea.
2. **Entities** (`entities/`) — things with proper names. People, teams, organizations, products, vendors. Named by the entity.
3. **Sources** (`sources/`) — specific papers, books, articles, talks, meeting notes. Named by a slug for the source.

Lifecycle: expanded frequently, rarely deleted. Each new source contributes facts to existing concept/entity notes and may create new ones.

Distinguishing property: semantic memory is about recall. If you asked yourself "what do I know about X?", the answer comes from semantic memory.

Episodic memory — WHAT HAPPENED

Contents: the log of activity, what you did, what was found, what worked, what failed. Chronological.

Examples: - "2026-04-09 AM — completed the platform migration RFC review, identified three concerns, sent to team" - "2026-03-15 — decided to extend Datadog contract rather than switch to New Relic (see decisions/2026-03-15-datadog-extend.md)"

Lifecycle: append-only short-term. Compressed weekly into summaries. Archived quarterly.

Where it lives: `sessions/` (running log), `decisions/` (significant decision records), `retros/` (retrospectives).

Distinguishing property: episodic memory is about temporality. When something happened matters. The answer to "when did I first notice this issue?" lives in episodic memory.

Working memory — CURRENT DRAFTS

Contents: half-formed ideas, exploratory writing, drafts that aren't ready to be atomic notes.

Examples: - A five-paragraph rumination on a talk you attended, which might later become two concept notes and an entity note - An email reply you're drafting, pulling from several wiki notes - A brain-dump from a meeting you need to process later

Lifecycle: short. Either promoted to canonical wiki content (once mature) or deleted (once obsolete). Nothing stays in working memory permanently.

Where it lives: `sandbox/`. A safe place. Freely deleted.

Distinguishing property: working memory is about liminality. It is not yet canonical; it is not yet discarded. It is in transit.

Why the typology matters

The four types differ in three orthogonal ways:

Type	Lifecycle	Editor	Read-frequency
Procedural	Slow-changing	You + LLM propose	Always-loaded
Semantic	Expanding	Both	JIT-loaded
Episodic	Append-compress-archive	Mostly you	Rarely read in full
Working	Transient	You mostly	Rarely read by LLM

When a wiki starts to rot, it is almost always because types are getting mixed. Session notes leak into `concepts/`. Drafts get published as canonical. Episodic content gets treated as semantic. Procedural rules get buried in random notes where the LLM cannot find them.

Enforcing the separation — which folder holds which type — is how you keep the wiki healthy.

Part VI — The technology stack, layer by layer

The wiki is, at bottom, a folder of markdown files. This simplicity is a feature. Every technology choice in the stack is in service of keeping the wiki portable, human-readable, and version-controllable.

Markdown as the substrate

Markdown wins as the content format for five reasons:

1. **Human-readable.** You can read a markdown file in any text editor, any terminal, any web browser. No proprietary format. No lock-in.
2. **LLM-readable.** Every modern LLM is trained heavily on markdown. It is effectively the LLM-native serialization format for structured human knowledge.
3. **Toolable.** `grep`, `sed`, `awk`, `ripgrep`, `find`, Python, Node, shell scripts — all work natively on markdown. You can write arbitrary maintenance tooling in whatever language you know.
4. **Diffable.** Git's line-based diff works perfectly on markdown. You can see what changed in any note over time.
5. **Portable.** You can move the wiki to a different computer, a different OS, a different editor, a different LLM. The content comes with you.

The alternatives — proprietary formats, databases, block-based JSON (Notion), binary formats — all fail at one or more of these properties. Markdown is not an accidental choice; it is the load-bearing format decision.

Obsidian as the frontend

Obsidian is the recommended editor for one reason: **it is a view over a folder of markdown files, not a container.** When you point Obsidian at your wiki folder, it does not move your files, does not lock them, does not embed them in a proprietary database. The files remain plain markdown on disk. Obsidian merely provides a rich editing and browsing experience over them.

What Obsidian adds:

- **Graph view** — visualize the network of cross-references as a 2D graph. Clusters emerge; orphans are visible.

- **Backlinks pane** — for any note, see every other note that references it. Bidirectional navigation.
- **Link autocomplete** — type `[[` and get suggestions from your vault.
- **Instant full-text search.**
- **Tag browser** — see all tags and their notes hierarchically.
- **Plugins** — free ecosystem including Dataview (SQL-like queries over notes), Templater (snippets), Omnisearch (semantic search), Graph Analysis (advanced graph metrics).
- **Cross-platform** — Mac, Windows, Linux, iOS, Android. The same vault on every device.

Crucially, Obsidian is free for personal use, has no mandatory account, sends no data anywhere by default, and is fully functional offline.

Alternatives exist — Logseq (outline-first), Dendron (VS Code plugin), plain editors (VS Code, Sublime, Vim) — but Obsidian is the sweet spot for the Karpathy pattern. It supports atomic notes and dense cross-references natively, has a mature plugin ecosystem, and does not impose opinions on how you organize.

Claude (or any LLM) as the maintainer

The LLM is the wiki's clerk. It performs:

- **Ingestion** — reading raw sources, extracting concepts and entities, writing new atomic notes, extending existing ones, updating the index and log
- **Query** — reading relevant wiki files in response to a question, answering with citations
- **Lint** — periodic maintenance: finding orphans, broken links, gaps, contradictions; compressing session entries; regenerating the index
- **Promotion** — when a conversation produces a synthesis that belongs in the wiki, proposing that it be written back

Claude Opus 4.6 with 1M context is the current recommendation for ingestion and lint — tasks that may need to read many files at once. Sonnet 4.6 is sufficient for most queries. Haiku is fine for lightweight lookups.

The LLM is not autonomous. It proposes; you approve. The boundary is critical: the LLM is very good at clerical synthesis on text, and not yet good enough to be trusted with unilateral authority over your knowledge.

Git for version control

Wrapping the wiki in a Git repository is a small additional step with large benefits:

- **History** — every change is tracked. You can see what the wiki looked like six months ago. You can roll back a bad ingestion.
- **Backup** — pushing to a remote (private GitHub repo, GitLab, self-hosted) is the simplest possible backup strategy. If your disk dies, the wiki survives.
- **Branching for experiments** — large structural changes (re-organizing folders, introducing new conventions) can happen on branches and be merged only if they work.
- **Audit** — the commit history is another audit trail, complementary to `log.md`.

Git is optional. It is strongly recommended. For a work wiki, it is nearly mandatory (IT policy permitting) because backup is non-negotiable for professional knowledge.

Semantic search as a navigation aid

For wikis under ~100 files, the index alone is sufficient navigation. The LLM reads the index, picks files, reads them.

For wikis larger than ~100 files, semantic search becomes useful — as a navigation aid, not as a retrieval replacement. The pattern:

- Embed every wiki file (not chunks — whole files) with an embedding model
- Store embeddings in SQLite or a small local vector DB
- At query time, compute the query embedding and return the top-k most similar *files* (not chunks)
- The LLM reads those files in full

This is file-level semantic retrieval. It preserves structural context. It does not fragment. It is compatible with Karpathy's "no RAG" doctrine in spirit, because retrieval happens at the file level and the LLM still reads whole files.

Embedding provider options:

- **Local** — `nomic-embed-text` via `llama.cpp` or Ollama, `sentence-transformers` in Python. Free, private, works offline.
- **OpenAI** — `text-embedding-3-small`, about $0.02 per million tokens. Cheap, high quality, cloud.
- **Voyage AI** — retrieval-specialized, slightly more expensive, higher quality for retrieval tasks.
- **Cohere** — another strong option.

For personal wikis under 1,000 notes, the cost difference between providers is negligible — we are talking about dollars per year. The real choice is privacy: if the wiki contains anything sensitive, prefer local embeddings.

MCP (Model Context Protocol) servers for data flow

MCP, Anthropic's open protocol for connecting LLMs to tools and data sources, is the integration layer for a live wiki. MCP servers let the LLM reach into external systems without custom API code per tool.

Relevant MCP servers for a second brain:

- **Filesystem MCP** — read and write files in the wiki
- **Gmail MCP** — ingest interesting emails as raw sources
- **Slack/Teams MCP** (where available) — capture conversations
- **GitHub MCP** — pull in code-related knowledge
- **Web fetch / search MCP** — research new topics
- **Calendar MCP** — pull meeting context into session notes

Not every MCP server needs to be wired into every wiki. You add integrations as they become useful. The MCP ecosystem is young and growing; the set of available servers in 2026 is much richer than a year ago.

The whole stack

Putting it together, a mature LLM-native second brain has this stack:

```
┌──────────────────────────────────────────────────────┐
│  Claude (Opus / Sonnet / Haiku)                      │   ← the maintainer
│  via Claude Code or Claude Desktop                   │
└──────────────────────────────────────────────────────┘
                        │
                        │  reads / writes via MCP
                        ↓
┌──────────────────────────────────────────────────────┐
│  Obsidian                                            │   ← the human frontend
│  (Dataview, Templater, Omnisearch plugins)           │
└──────────────────────────────────────────────────────┘
                        │
                        │  edits
                        ↓
┌──────────────────────────────────────────────────────┐
│  Local markdown files                                │   ← the substrate
│  wiki/ directory with index.md, log.md, etc.         │
└──────────────────────────────────────────────────────┘
                        │
                        │  versioned by
                        ↓
┌──────────────────────────────────────────────────────┐
│  Git repository (local + private remote)             │   ← the backup
└──────────────────────────────────────────────────────┘
                        │
                        │  optionally indexed by
                        ↓
┌──────────────────────────────────────────────────────┐
│  Local SQLite + local embeddings                     │   ← the search aid
│  (only at scale; skip for <100 files)                │
└──────────────────────────────────────────────────────┘
```

This stack is remarkably simple. Every layer is commodity technology. Every layer is human-inspectable. No vendor lock-in at any point.

Part VII — The ingest-query-lint operational loop

The wiki is not a static artifact. It is a living system operated through three recurring processes. Understanding these three processes is how you understand the wiki's daily life.

Ingest — content flows in

Ingestion is how material from Layer 1 (raw sources) becomes Layer 2 (wiki content). It is the most expensive and most important of the three processes.

The general workflow:

1. **Capture** — you drop a file into `raw/`. Paper, article, PDF, markdown export, text, anything.
2. **Classify** — the LLM reads the first portion and determines what kind of content it is (paper, article, meeting notes, book excerpt, conversation log, etc.).
3. **Extract** — the LLM identifies the key concepts and entities mentioned.
4. **Check** — for each concept or entity, the LLM checks whether a wiki page already exists.
5. **Write or extend** — for existing pages, add a new section with content from this source. For new concepts or entities, create a new atomic note.
6. **Cite** — every new fact cites its source. Every section added to an existing page cites the source.
7. **Create source note** — write a note about the source itself into `sources/`, with bibliographic details, a one-paragraph summary in your voice, and a list of the concept/entity notes the source contributed to.
8. **Move** — the raw file moves from `raw/` to `raw/processed/{date}/`. Never deleted.
9. **Update navigation** — new entries added to `index.md`; an ingest entry appended to `log.md`.
10. **Report** — the LLM summarizes what was ingested, what was created, what was extended, and flags anything uncertain for your review.

The critical rule: paraphrase, never transcribe. Write every new fact in your own voice. Never quote more than one sentence from a source. The whole value of the wiki depends on this rule. Violate it and the wiki becomes a pile of quotations — a decorated bookmark folder — instead of a compiled understanding.

The second critical rule: extend, don't overwrite. If a concept note exists, add to it; do not replace what's there. The wiki's value is cumulative; destroying past content to make room for present content is the antithesis of compounding.

Query — questions get answered

Query is how the wiki gets used. Every conversation in Claude — every question about your knowledge — is a query.

The general workflow:

1. **Read the schema** — `wiki/README.md`, always loaded at session start.
2. **Read the index** — `wiki/index.md`, to see what pages exist.
3. **Identify relevant files** — based on the query, list the concept, entity, source, project, and session files likely to contain relevant content.
4. **Read the files in full** — not chunks, not summaries, the whole files.
5. **Synthesize** — draw from the files to answer the query, citing each source file.
6. **Propose promotion** — if the answer contains synthesis that goes beyond what's explicitly in the wiki (you connected two ideas across files; you added context from the LLM's general knowledge), ask whether to promote the synthesis back into the wiki as a new section.
7. **If approved, update** — write the new section to the appropriate file, append a query entry to `log.md`.

Query is the fastest-feedback loop in the wiki. Every question either confirms the wiki has what you need (retention) or reveals a gap (new content opportunity). Over time, queries teach you where your wiki is strong and where it needs work.

Lint — quality is maintained

Lint is the weekly hygiene process. Without linting, wikis rot. With it, they self-improve.

The general workflow, performed weekly:

1. **Inventory** — count files by category. Check that growth is happening where it should be.
2. **Find orphans** — files with neither inbound nor outbound links. Flag for review. Every orphan is either a file that should be linked in, or a file that should be archived.
3. **Find broken links** — `[[wikilinks]]` pointing to files that no longer exist. Auto-fix the obvious ones (renames, moves); flag the rest.
4. **Find gap candidates** — concepts mentioned across many notes without a dedicated atomic note. Propose creating one.
5. **Find contradictions** — entity or concept claims that conflict across notes. Flag them; leave resolution to you.
6. **Find stale notes** — files untouched for 90+ days. Flag for "still relevant?" review. Never auto-delete.

7. **Compress episodic memory** — move completed week from `sessions/current-week.md` into `sessions/weekly/YYYY-W##.md`.
8. **Regenerate index** — scan all files, rebuild `index.md` from current state with one-line summaries alphabetized within each section.
9. **Generate maintenance report** — what was auto-fixed, what needs your review.
10. **Append to log** — a lint entry summarizing the run.

The hard rule of lint: **propose, don't act** on anything touching canonical concept, entity, source, or project content. The lint task can regenerate mechanical things (the index, compressed session files, broken links that are obviously renames). Anything else is a suggestion for your Friday afternoon review.

Why weekly, not daily? A week is the right rhythm for patterns to emerge. Daily lint is noisy; monthly is too slow to catch drift. Weekly is when the wiki has had enough activity to reveal structure and not so much that the report becomes overwhelming.

The three-process rhythm

Taken together, ingest-query-lint forms the wiki's heartbeat:

- **Ingest** happens whenever you have raw material. Weekly is a reasonable cadence, or on-demand when you've collected enough to process.
- **Query** happens continuously — every Claude conversation is a potential query.
- **Lint** happens weekly, ideally at a fixed time (Friday afternoon as a "close the week" ritual, or Monday morning as a "open the week" ritual).

The rhythm is not about productivity. It is about keeping the wiki healthy. A wiki that is ingested regularly, queried often, and linted weekly is a wiki that will compound. A wiki that is ingested in a burst, queried sporadically, and never linted will rot within a year.

Part VIII — Why this works now and didn't before

A reasonable skeptic could ask: if the ideas behind the second brain are a century old, and the technology has been "good enough" for years, why is this pattern only working now? The answer has four parts. Each part is a recent shift; together, they explain why 2025 was the inflection.

The long-context shift

Through 2022, LLM context windows were measured in thousands of tokens. GPT-3 was 2,048. Early GPT-3.5 was 4,096. These windows were too small to hold a meaningful knowledge base. RAG was the only option — you had to chunk and retrieve because you could not fit a useful corpus.

By late 2024, Claude 3.5 Sonnet and GPT-4 Turbo had 128K–200K context windows. A personal wiki could now fit. But the pattern was not obvious yet; the field was still paradigm-locked on RAG.

By 2025, Claude Opus 4 launched with 200K and later extended to 1M. Gemini 2.5 launched with 1M. OpenAI's o-series had similar capacities. A 400,000-word wiki fits comfortably in 1M context. The RAG-first paradigm became a choice, not a necessity.

This is the central enabler. Without long context, the Karpathy pattern is impossible. With long context, the whole RAG infrastructure becomes optional.

The synthesis capability shift

Long context is necessary but not sufficient. A model that can hold 1M tokens but synthesizes poorly across them is not useful. The second shift was the dramatic improvement in cross-document synthesis that came with Opus 4, GPT-5, Gemini 2.5 Pro, and comparable frontier models in 2025.

These models can read 50 related files and produce a coherent synthesis that accurately reflects all of them. They can spot contradictions. They can fill gaps. They can paraphrase in a consistent voice across a long session. Previous models could do these things weakly; frontier 2025 models do them routinely.

This is why the wiki's value is not just "storage + search" but "storage + search + on-demand synthesis." The synthesis is where the compounding happens. Without it, the wiki is just an organized archive. With it, the wiki becomes a cognitive partner.

The agent-harness shift

Until mid-2025, LLM applications were mostly single-turn: one prompt in, one response out. Complex workflows required custom orchestration code (LangChain, CrewAI, custom harnesses). This was a high barrier to sophisticated tool use.

The arrival of Claude Code, Claude Desktop with agent capabilities, Anthropic's Managed Agents platform, OpenAI Assistants / Responses, and the MCP ecosystem changed this. An agent can now read files, execute code, search the web, draft updates, and iterate — all within a single session, without custom orchestration.

This matters for the wiki because ingestion is an agentic task. The LLM must read a source, check existing files, create new files, update multiple files, move raw files to archive, and update navigation files. Ten years ago this would have required a custom pipeline. Now it's a SKILL.md prompt and a capable model.

The obsidian-stack shift

Obsidian launched in 2020 and matured rapidly through 2022–2024. By 2025, it was the clear winner as a markdown-native personal knowledge editor. The plugin ecosystem was rich; the graph view was mature; it worked across all major platforms; it was free.

This mattered because the wiki needs a human frontend. Without Obsidian (or an equivalent), the wiki is markdown files you can't browse pleasantly. With Obsidian, you have graph views, backlinks, instant search, link autocomplete, and a pleasant editing experience over the same plain files the LLM reads.

The combination of Obsidian + plain markdown + LLM maintainer is what makes the modern second brain feel like a single coherent system rather than three disjoint tools stapled together.

Why not five years ago

Five years ago, this pattern was impossible for four reasons:

1. Context windows too small (RAG was the only option)
2. Synthesis too weak (the LLM could retrieve but not really think across documents)
3. No agent harnesses (building the ingest pipeline required custom code, expensive and fragile)
4. No mature markdown-native editor with Obsidian's feature set

By 2025, all four were addressed. The pattern emerged naturally from the conditions. Karpathy was the articulator, but the design space had converged.

Part IX — Comparison to alternatives

Understanding why the LLM wiki works requires understanding what it displaces.

vs traditional note-taking (Apple Notes, OneNote, Evernote)

Traditional note apps are write-only systems. You capture notes; you rarely reopen them. Retrieval is via full-text search, which requires you to remember what you wrote. The notes accumulate without compounding — each new note is isolated from the others.

The LLM wiki inverts this. Notes are heavily linked; retrieval is both full-text and semantic (via the index). The LLM cross-references across notes. The notes compound.

When traditional notes are fine: ephemeral capture, shopping lists, quick reminders, anything you don't need to connect to other knowledge.

When the LLM wiki is strictly better: any content you might want to synthesize across or retrieve months later.

vs modern notes apps (Notion, Roam Research, Logseq)

These tools improved the substrate — block-level references, backlinks, graph views, databases. But the human is still the maintainer. Writing a note and keeping the graph healthy both fall on the user.

The LLM wiki splits the labor. You do the creative labor (capture, insight, judgment). The LLM does the clerical labor (cross-references, index updates, gap detection, contradiction spotting).

When Notion is fine: collaborative wikis where multiple humans contribute and conventions are enforced by team norms.

When Roam is fine: single-user intensive knowledge work by someone who genuinely enjoys graph maintenance.

When the LLM wiki is strictly better: single-user knowledge work by someone who does not want to spend an hour a day on maintenance.

vs chat-with-your-docs products (Glean, Guru, Notion AI Q&A)

These products sit on top of your existing documents and provide a chat interface to query them. They typically use RAG under the hood.

The fatal issue: the knowledge stays in its original form. Nothing is compiled. The product cannot improve the organization of the source material; it can only search it. And search over poorly-organized content returns poorly-contextualized results.

When chat-with-your-docs is fine: a one-shot question against a fixed corpus you don't own (a technical manual, a legal codex, a documentation set).

When the LLM wiki is strictly better: any knowledge base you own and want to evolve.

vs pure RAG systems

Pure RAG treats every query as independent retrieval. Chunks in; answer out; nothing persists.

This works when the corpus is truly static and the queries are truly isolated. Most real knowledge work violates both assumptions. Corpuses evolve; queries build on prior queries.

When RAG is fine: customer support over a fixed product manual, legal research against a fixed body of law, medical literature search where the corpus is vast and queries are discrete.

When the LLM wiki is strictly better: personal or team knowledge work where the corpus and queries co-evolve.

vs graph databases and knowledge graphs

Formal knowledge graphs (triples, RDF, Neo4j) were the previous-generation attempt at structured knowledge. They required rigid schemas — every fact had to fit a predefined predicate. Maintaining the schema was its own project.

The LLM wiki is a soft knowledge graph. The "nodes" are markdown files. The "edges" are `[[wikilinks]]`. There is no enforced schema; the conventions emerge. The LLM can reason about the graph without needing it to be formalized.

When formal knowledge graphs are fine: enterprise integration where machine-to-machine reasoning requires rigid schemas.

When the LLM wiki is strictly better: human-in-the-loop knowledge work where flexibility beats formality.

Part X — The compounding value thesis

The strongest argument for the LLM-native second brain is not that it solves today's problems. It is that its value grows non-linearly over time. This part makes the compounding thesis explicit.

Linear vs compounding systems

Most note-taking systems are **linear** in value: a note added today is worth approximately what a note added a year ago is worth. Total value is proportional to total notes.

A compounding system's value grows faster than its content. Each new note connects to many existing notes, which creates new accessibility patterns for the whole graph. The 500th note makes the first 499 notes more valuable, because now the LLM has more context to synthesize across.

The distinguishing feature of a compounding system is **dense cross-linking combined with active synthesis.** Cross-linking without synthesis is just a graph. Synthesis without cross-linking is isolated smart answers. The combination — a graph the LLM can synthesize across — is what compounds.

The mathematical argument

Model the wiki's value as a function of its notes:

- **Linear:** $V(n) = k \times n$ — each note adds constant value
- **Network:** $V(n) = k \times n^2$ — each note adds value proportional to existing notes (Metcalfe's law analog)
- **Karpathy wiki:** $V(n) \approx k \times n \times \log(n)$ — each note adds value proportional to how well the LLM can synthesize across existing notes, which grows sub-linearly but steadily

The actual growth function is empirical — it depends on writing discipline, linking density, synthesis quality — but the shape is super-linear. A wiki of 1,000 notes is not 10× more valuable than a wiki of 100; it is more like 30× to 50× more valuable, because the synthesis surface is much richer.

This is the compounding thesis: **the wiki's value grows faster than your investment in it, provided you maintain the linking and synthesis discipline.**

The maintenance thesis

The corresponding claim about maintenance cost:

- **Linear systems:** maintenance cost grows proportionally with size. Every new note requires marginal tagging, linking, index update work.
- **LLM-maintained systems:** maintenance cost is approximately constant regardless of size. The LLM handles the clerical work; only your review time scales, and only linearly (more content = more to review).

This is the crucial asymmetry. In traditional PKM, value grows linearly but maintenance grows with size, eventually the maintenance catches up and the system rots. In LLM-maintained PKM, value grows super-linearly while maintenance stays roughly constant. The system never rots — it only improves.

Whether this is true in practice is an empirical question. Current evidence (Karpathy's own wiki, Matuschak's evergreen notes, and an emerging cohort of production wikis across creative and engineering domains) suggests it holds at least through the first year. Longer-term validation requires longer-term data.

The human-in-the-loop check

One failure mode could break the compounding: LLM drift. If the LLM's paraphrasing gradually diverges from your voice, or if the LLM's synthesis introduces subtle errors, the wiki accumulates low-quality content that pollutes future synthesis.

The safeguard is the weekly review. You read what the LLM wrote; you correct drift; you reject bad syntheses. The wiki is as good as your review discipline.

In practice, the review takes 20–30 minutes per week. This is the price of compounding. If you skip it, you lose the compounding. If you maintain it, you get compounding for a cost that stays constant while value grows.

Case studies

Several public or semi-public LLM wikis are now in their second year of operation. Below, two are publicly verifiable; two are composite illustrations drawn from early production use across creative and engineering domains — clearly marked as such and included because the pattern's applicability beyond the original Karpathy use case is itself load-bearing evidence.

- **Karpathy's own wiki** *(publicly documented)* — described in his gist, approximately 100 articles and growing. He reports it is the most valuable personal productivity tool he has ever built.
- **Matuschak's evergreen notes** *(publicly readable at notes.andymatuschak.org)* — predates LLM maintenance but exemplifies the structural discipline. Thousands of atomic notes, densely linked, published as a living digital garden.
- **An independent electronic musician's catalog wiki** *(composite illustration)* — a solo producer building a multi-album back catalog uses an LLM-maintained wiki to track ~200 finished tracks, ~400 visual assets, release metadata across five platforms, and production notes (synth patches, vocal treatments, mastering decisions, album concept arcs). The wiki has been running continuously since early 2026. Ingest processes new Suno exports, Midjourney outputs, and session notes; lint runs weekly; entity notes for recurring visual characters and sonic motifs cross-reference back to every track that uses them. The producer reports that album-over-album release operations time dropped from roughly 18 hours to roughly 3 hours once the wiki's release-orchestration patterns matured, with the majority of the savings attributable to the wiki's ability to instantly surface "what did we do for the last release that worked."
- **An enterprise product engineering team's technical knowledge base** *(composite illustration)* — a ~20-person product engineering organization runs an LLM-maintained wiki covering platform architecture, vendor evaluations, Architectural Decision Records (ADRs), quarterly planning artifacts, and post-incident retrospectives. Running since early 2026. A dedicated agent runs weekly lint; team members contribute atomic notes via the query-promotion pattern (good answers from Claude conversations get promoted back into the wiki). The team reports onboarding time for new engineers dropped from roughly 4 weeks to roughly 1.5 weeks, attributed largely to the wiki's ability to answer "why did we decide X?" questions instantly, with full cited context, for any architectural decision made in the last 18 months.
- **Various "building in public" knowledge systems** *(publicly documented)* — individuals publishing their LLM wikis as digital gardens, using Quartz or Obsidian Publish. Independent implementations such as the `wiki-skills` Claude Code plugin (github.com/kfchou/wiki-skills) and the `llm-wiki-compiler` (github.com/ussumant/llm-wiki-compiler) provide working reference code.

The common pattern across these case studies: the wiki becomes indispensable within 4–6 months. Before that, it is marginal. After that, the user cannot imagine going back.

Part XI — Open questions, critiques, and frontiers

The LLM-native second brain is new. It is not a solved problem. Several open questions and legitimate critiques deserve airing.

Will it actually compound long-term?

The compounding thesis rests on the assumption that LLM-maintained wikis do not accumulate subtle errors faster than they accumulate value. We have one year of production data. We do not have ten years.

Risk scenarios:

- **Quiet drift in paraphrasing** — the LLM's voice slowly diverges from yours
- **Accumulated small errors** — a misparaphrased claim becomes a source of confusion in future notes
- **Ossification of wrong decisions** — an early choice about structure becomes hard to reverse after a thousand notes

Mitigations: weekly review, version control via Git, periodic deep audits (quarterly), willingness to do structural refactors. The wiki is a living system; it must evolve its own conventions.

What about vendor lock-in to LLM providers?

The LLM-native second brain depends on access to a capable long-context model. Today that means Claude, GPT, Gemini, or a comparable frontier model. If those providers change pricing, policies, or capabilities adversely, the wiki's ongoing operation is at risk.

Mitigations:

- **The content is portable.** Plain markdown files. No lock-in to any provider. You can switch the maintainer LLM at any time.
- **Multiple providers work.** The pattern does not depend on any specific LLM. Any model with 200K+ context and good synthesis can maintain the wiki.
- **Self-hosted options are improving.** Open-weight models in 2026 are approaching frontier capability for many tasks. A fully self-hosted LLM wiki is feasible today and will get easier over time.

The lock-in is to the *pattern*, not to any specific provider. This is a much weaker form of lock-in than proprietary note apps.

What about collaboration?

The Karpathy pattern is single-user by design. Two humans cannot maintain a wiki the same way one can, because the paraphrasing-in-your-voice rule breaks down.

Approaches to collaboration:

- **Team Notion/Confluence for shared knowledge, personal LLM wiki for individual knowledge.** Keep them separate. Export from the personal wiki to the team system when something becomes shareable.
- **Shared wiki with per-section attribution.** Each person's sections use their voice; the wiki has an implicit author per page or per section.
- **Wiki-of-wikis federation.** Each team member has their own LLM wiki; a shared index links across them.

None of these is as clean as the single-user pattern. Collaboration is a frontier for LLM-native PKM.

What about privacy and security?

A detailed second brain contains sensitive material. Even work-only wikis include internal politics, personnel observations, decision rationale, stakeholder opinions. The wiki is not just knowledge; it is also a high-trust artifact.

Risks:

- **LLM provider access** — if the wiki content is sent to an LLM for processing, the provider sees it. Anthropic, OpenAI, Google all have published data handling policies; read them.
- **Local disk theft** — a compromised laptop exposes the whole wiki. Encrypt the disk at minimum.
- **Embedding provider leak** — if you use cloud embeddings, those providers see the content.
- **Git remote compromise** — if you back up to a Git remote, that remote becomes an attack surface. Private repos on trusted hosts mitigate but don't eliminate.

Mitigations:

- Disk encryption (standard on modern Macs; enable FileVault)
- Zero-retention agreements with LLM providers when available
- Local embeddings when privacy matters

- Private Git remotes, ideally self-hosted or from a trusted enterprise provider
- Separate wikis for different sensitivity tiers (e.g., personal / work / highly-sensitive)

Privacy is not a solved problem. It is a continual design choice.

Is paraphrasing actually better than quoting?

The "paraphrase, never transcribe" rule is load-bearing in this document. But it has a cost: if the original wording is important (because of legal precision, scientific specificity, poetic quality), paraphrasing destroys information.

The answer is nuanced: paraphrase is the default for concepts and entities. Direct quotation is fine when the wording itself is the point — a specific legal phrase, a memorable formulation, a primary-source artifact. The wiki can mix both. The rule is that the *default* is paraphrase; exceptions are explicit.

Are we externalizing too much?

A final, more philosophical critique: if the LLM maintains your second brain, are you losing the benefit of maintenance-as-learning? The old saying in classical Zettelkasten practice was that the bookkeeping itself was where understanding happened.

The counter-argument: clerical bookkeeping is not where understanding happens. Understanding happens when you paraphrase a source in your own words — the act of compression into your voice. The LLM does not replace that act; it only handles the downstream clerical work (cross-references, index updates) that does not contribute to understanding anyway.

In practice, LLM-maintained wikis tend to surface more opportunities for paraphrasing than traditional wikis, because the LLM does the clerical work that would otherwise consume your attention. You end up doing more creative labor, not less.

But this is an empirical claim, and individuals will vary. Some people learn best through maintenance. The pattern is not universally right.

Closing thoughts

The LLM-native second brain is the first personal knowledge management pattern in history that plausibly escapes the maintenance trap. It does so not by making maintenance cheaper — traditional PKM has been trying to do that for decades, with limited success — but by shifting the mainte-

nance to an agent that genuinely does not get bored and does not get worse over time.

It is early. The pattern is one year old in production. We do not yet know what it looks like at five years, at ten years, at a lifetime. Some of the design choices will turn out to be wrong; some will turn out to be more load-bearing than we realized.

But the core insight is robust: **if you structure your knowledge as a human-readable markdown wiki, if you enforce atomic notes and dense cross-references, if you run ingest-query-lint on a weekly cadence, and if you review your LLM's work regularly, you get a personal knowledge system that compounds in value over time at a maintenance cost that stays roughly constant.**

This is a structural change in what knowledge work can be. The question is no longer "how do I keep my notes organized?" It is "what do I now choose to learn, given that the compounding is no longer blocked by bookkeeping?"

That is a better question to be working on.

Selected sources and further reading

Karpathy's primary writings: - Karpathy, LLM Wiki gist — gist.github.com/karpathy/442a6bf555914893e9891c11519de94f - Karpathy on context engineering, June 2025 — x.com/karpathy/status/1937902205765607626 - Karpathy, 2025 Year in Review — karpathy.bearblog.dev/year-in-review-2025/

Anthropic engineering: - Effective Context Engineering for AI Agents, Anthropic, September 2025 — anthropic.com/engineering/effective-context-engineering-for-ai-agents

Foundational PKM: - Vannevar Bush, *As We May Think*, The Atlantic, 1945 - Doug Engelbart, *Augmenting Human Intellect*, SRI, 1962 - Niklas Luhmann, papers on the Zettelkasten method - Andy Matuschak, notes.andymatuschak.org (especially the evergreen-notes sequence) - Maggie Appleton, maggieappleton.com/garden (digital gardens) - Tiago Forte, *Building a Second Brain*, 2022

Implementations and case studies: - Obsidian — obsidian.md - Quartz (Obsidian digital garden) — quartz.jzhao.xyz - Claude-Blueprints library (Phil Mora + Claude, 2026)

End of document.